

SOFTWARE REQUIREMENT SPESIFICATION (SRS)

Nurfitria Khoirunnisa

PENGERTIAN

Software Requirement Spesification (SRS) / Spesifikasi Kebutuhan Perangkat Lunak (SKPL)

Adalah sebuah dokumen yang berisi pernyataan lengkap dari apa yang dapat dilakukan oleh perangkat lunak, tanpa menjelaskan bagaimana hal tersebut dikerjakan oleh perangkat lunak

Adalah dokumen yang bertujuan untuk memberikan deskripsi yang komprehensif tentang produk perangkat lunak yang akan dikembangkan, termasuk tujuannya, proses bisnisnya, fitur, parameter kinerjanya, dan perilaku. Fungsinya -> Peta yang memandu proses pengembangan dan menjaga semua orang di jalur yang benar

MANFAAT

Penggunaan SRS dapat menghilangkan dan mencegah kesalahan dalam fase desain karena persyaratan dan fungsi yang bertentangan yang memerlukan validasi dapat diperbaiki pada saat ini dan pemangku kepentingan dapat untuk evaluasi ulang.

FUNGSI

1. Mencatat semua kebutuhan calon pengguna Perangkat Lunak (PL).
2. Sebagai kontrol saat proses pengembangan PL dilakukan, sehingga setiap tahapan pengerjaan pengambangan sesuai dengan yang diharapkan
3. Digunakan sebagai acuan pada saat pengujian dilakukan sehingga hasil akhir sesuai dengan yang dibutuhkan
4. Dijadikan pedoman jika terdapat perbedaan pendapat antara calon pemakai dengan pengembang sistem terhadap hasil dari pengembangan PL
5. Bukti bahwa pengembang telah melakukan tahap software requirement analysis.

ATRIBUT PENULISAN SRS YANG BAIK

SRS yang baik harus memenuhi beberapa karekeristik utama. Harus :

1. Benar (Correct)
2. Tepat (Precise)
3. Unambigouity
4. Lengkap (Complete)
5. Dapat diverifikasi (Verifiable)
6. Konsisten
7. Understandable
8. Dapat dimodifikasi (Modifieadable)
9. Dapat dilacak sumbernya (Traceable)

YANG HARUS DIHINDARI KETIKA PEMBUATAN SRS

1. Over specification (penjelasan berlebihan dan berulang-ulang sehingga menjadi tidak jelas
2. Tindakan unconcistency
3. Ambiguity dalam kata atau kalimat
4. Menuliskan “mimpi-mimpi”, yaitu hal yang tidak bisa dilakukan

REVISION HISTORY

Revision History

Author	Date	Description	Version

1	0	0
MAJOR	MINOR	REVISION

MAJOR : Perubahan besar, misal struktur dokumen berubah total, atau isi utama berubah signifikan
MINOR : Penambahan isi baru, sub-bab, tambahan fitur dalam dokumen (fitur konten)
REVISION : Perbaikan kecil pada bagian minor tersebut, seperti typo, format, penyesuaian minor (bukan isi besar)

Revision History

Author	Date	Description	Version
Andi	2025-08-01	Dokumen awal dibuat dan disusun lengkap	1.0.0
Andi	2025-08-10	Ditambah bab 2: Prosedur penggunaan baru	1.1.0
Budi	2025-08-12	Perbaikan typo dan format bab 2	1.1.1
Cucu	2025-08-20	Penambahan sub-bab 3.1 tentang kebijakan	1.2.0
Andi	2025-08-25	Revisi besar: penataan ulang bab dan isi	2.0.0
Cucu	2025-08-27	Koreksi minor pada daftar isi dan footer	2.0.1

KOMPONEN SRS SESUAI IEEE

Pendahuluan

a. Tujuan Dokumen
b. Ruang Lingkup Produk
c. Definisi, Akronik, dan Singkatan
d. Referensi
e. Sistematika

Deskripsi Umum Perangkat Lunak

a. Deskripsi Umum PL
b. Fungsi Produk
c. Karakteristik Pengguna
d. Batasan-batasan
e. Asumsi dan Ketergantungan

Boleh disesuaikan kembali dengan versi lain di standar IEEE

Versi 1

3. Specific requirements
3.1 External interface requirements
- 3.1.1 User interfaces
- 3.1.2 Hardware interfaces
- 3.1.3 Software interfaces
- 3.1.4 Communications interfaces

3.2 Functional requirements
- 3.2.1 Mode 1
  - 3.2.1.1 Functional requirement 1.1
  - .
  - .
  - .
  - 3.2.1.*n* Functional requirement 1.*n*
- 3.2.2 Mode 2
- .
- .
- .
- 3.2.*m* Mode *m*
  - 3.2.*m*.1 Functional requirement *m*.1
  - .
  - .
  - .
  - 3.2.*m*.*n* Functional requirement *m*.*n*

3.3 Performance requirements
3.4 Design constraints
3.5 Software system attributes
3.6 Other requirements

Versi 2

3. Specific requirements
3.1. Functional requirements
- 3.1.1 Mode 1
  - 3.1.1.1 External interfaces
    - 3.1.1.1.1 User interfaces
    - 3.1.1.1.2 Hardware interfaces
    - 3.1.1.1.3 Software interfaces
    - 3.1.1.1.4 Communications interfaces
  - 3.1.1.2 Functional requirements
    - 3.1.1.2.1 Functional requirement 1
    - 3.1.1.2.*n* Functional requirement *n*
  - 3.1.1.3 Performance
- 3.1.2 Mode 2
- .
- .
- .
- 3.1.*m* Mode *m*

3.2 Design constraints
3.3 Software system attributes
3.4 Other requirements

Versi 3

3. Specific requirements
3.1 External interface requirements
3.1.1 User interfaces
3.1.2 Hardware interfaces
3.1.3 Software interfaces
3.1.4 Communications interfaces
3.2 Functional requirements
3.2.1 User class 1
3.2.1.1 Functional requirement 1.1
.
.
.
3.2.1.*n* Functional requirement 1.*n*
3.2.2 User class 2
.
.
.
3.2.*m* User class *m*
3.2.*m*.1 Functional requirement *m*.1
.
.
.
3.2.*m*.*n* Functional requirement *m*.*n*
3.3 Performance requirements
3.4 Design constraints
3.5 Software system attributes
3.6 Other requirements

Versi 4

3. Specific requirements
3.1 External interface requirements
3.1.1 User interfaces
3.1.2 Hardware interfaces
3.1.3 Software interfaces
3.1.4 Communications interfaces
3.2 Classes/Objects
3.2.1 Class/Object 1

3.2.1.1 Attributes (direct or inherited)
3.2.1.1.1 Attribute 1
.
.
.
3.2.1.1.*n* Attribute *n*
3.2.1.2 Functions (services, methods, direct or inherited)
3.2.1.2.1 Functional requirement 1.1
.
.
.
3.2.1.2.*m* Functional requirement 1.*m*
3.2.1.3 Messages (communications received or sent)
3.2.2 Class/Object 2
.
.
.
3.2.*p* Class/Object *p*
3.3 Performance requirements
3.4 Design constraints
3.5 Software system attributes
3.6 Other requirements

Versi 5

3. Specific requirements
- 3.1 External interface requirements
  - 3.1.1 User interfaces
  - 3.1.2 Hardware interfaces
  - 3.1.3 Software interfaces
  - 3.1.4 Communications interfaces
- 3.2 System features
  - 3.2.1 System Feature 1
    - 3.2.1.1 Introduction/Purpose of feature
    - 3.2.1.2 Stimulus/Response sequence
    - 3.2.1.3 Associated functional requirements
      - 3.2.1.3.1 Functional requirement 1
      - .
      - .
      - .
      - 3.2.1.3.*n* Functional requirement *n*
  - 3.2.2 System feature 2
  - .
  - .
  - .
  - 3.2.*m* System feature *m*
  - .
  - .
  - .
- 3.3 Performance requirements
- 3.4 Design constraints
- 3.5 Software system attributes
- 3.6 Other requirements

Versi 6

3. Specific requirements
- 3.1 External interface requirements
  - 3.1.1 User interfaces
  - 3.1.2 Hardware interfaces
  - 3.1.3 Software interfaces
  - 3.1.4 Communications interfaces
- 3.2 Functional requirements
  - 3.2.1 Stimulus 1
    - 3.2.1.1 Functional requirement 1.1
    - .
    - .
    - .
    - 3.2.1.*n* Functional requirement 1.*n*
  - 3.2.2 Stimulus 2
  - .
  - .
  - .
  - 3.2.*m* Stimulus m
    - 3.2.*m*.1 Functional requirement *m*.1
    - .
    - .
    - .
    - 3.2.*m*.*n* Functional requirement *m*.*n*
- 3.3 Performance requirements
- 3.4 Design constraints
- 3.5 Software system attributes
- 3.6 Other requirements

3. Specific requirements
- 3.1 External interface requirements
  - 3.1.1 User interfaces
  - 3.1.2 Hardware interfaces
  - 3.1.3 Software interfaces
  - 3.1.4 Communications interfaces
- 3.2 Functional requirements
  - 3.2.1 Information flows
    - 3.2.1.1 Data flow diagram 1
      - 3.2.1.1.1 Data entities
      - 3.2.1.1.2 Pertinent processes
      - 3.2.1.1.3 Topology
    - 3.2.1.2 Data flow diagram 2
      - 3.2.1.2.1 Data entities
      - 3.2.1.2.2 Pertinent processes
      - 3.2.1.2.3 Topology
    - .
    - .
    - .
    - 3.2.1.*n* Data flow diagram *n*
      - 3.2.1.*n*.1 Data entities
      - 3.2.1.*n*.2 Pertinent processes
      - 3.2.1.*n*.3 Topology
  - 3.2.2 Process descriptions
    - 3.2.2.1 Process 1
      - 3.2.2.1.1 Input data entities
      - 3.2.2.1.2 Algorithm or formula of process
      - 3.2.2.1.3 Affected data entities
    - 3.2.2.2 Process 2
      - 3.2.2.2.1 Input data entities
      - 3.2.2.2.2 Algorithm or formula of process
      - 3.2.2.2.3 Affected data entities
    - 3.2.2.*m* Process *m*
      - 3.2.2.*m*.1 Input data entities
      - 3.2.2.*m*.2 Algorithm or formula of process
      - 3.2.2.*m*.3 Affected data entities
  - 3.2.3 Data construct specifications
    - 3.2.3.1 Construct 1
      - 3.2.3.1.1 Record type
      - 3.2.3.1.2 Constituent fields
    - 3.2.3.2 Construct 2
      - 3.2.3.2.1 Record type
      - 3.2.3.2.2 Constituent fields
    - .
    - .
    - .
    - 3.2.3.*p* Construct *p*
      - 3.2.3.*p*.1 Record type
      - 3.2.3.*p*.2 Constituent fields
  - 3.2.4 Data dictionary
    - 3.2.4.1 Data element 1
      - 3.2.4.1.1 Name
      - 3.2.4.1.2 Representation
      - 3.2.4.1.3 Units/Format
      - 3.2.4.1.4 Precision/Accuracy
      - 3.2.4.1.5 Range
    - 3.2.4.2 Data element 2
      - 3.2.4.2.1 Name
      - 3.2.4.2.2 Representation
      - 3.2.4.2.3 Units/Format
      - 3.2.4.2.4 Precision/Accuracy
      - 3.2.4.2.5 Range
    - 3.2.4.*q* Data element *q*
      - 3.2.4.*q*.1 Name
      - 3.2.4.*q*.2 Representation
      - 3.2.4.*q*.3 Units/Format
      - 3.2.4.*q*.4 Precision/Accuracy
      - 3.2.4.*q*.5 Range
- 3.3 Performance requirements
- 3.4 Design constraints
- 3.5 Software system attributes
- 3.6 Other requirements

3. Specific requirements
- 3.1 External interface requirements
  - 3.1.1 User interfaces
  - 3.1.2 Hardware interfaces
  - 3.1.3 Software interfaces
  - 3.1.4 Communications interfaces
- 3.2 Functional requirements
  - 3.2.1 User class 1
    - 3.2.1.1 Feature 1.1
      - 3.2.1.1.1 Introduction/Purpose of feature
      - 3.2.1.1.2 Stimulus/Response sequence
      - 3.2.1.1.3 Associated functional requirements
    - 3.2.1.2 Feature 1.2
      - 3.2.1.2.1 Introduction/Purpose of feature
      - 3.2.1.2.2 Stimulus/Response sequence
      - 3.2.1.2.3 Associated functional requirements
    - .
    - .
    - .
    - 3.2.1.*m* Feature 1.*m*
      - 3.2.1.*m*.1 Introduction/Purpose of feature
      - 3.2.1.*m*.2 Stimulus/Response sequence
      - 3.2.1.*m*.3 Associated functional requirements
  - 3.2.2 User class 2
  - .
  - .
  - .
  - 3.2.*n* User class *n*
  - .
  - .
  - .
- 3.3 Performance requirements
- 3.4 Design constraints
- 3.5 Software system attributes
- 3.6 Other requirements

3.1 Kebutuhan Fungsional

<Area ini menggambarkan pengorganisasian persyaratan fungsional untuk produk dengan fitur sistem, layanan utama yang disediakan oleh produk. Didefiinisikan dalam format user stories beserta acceptance criteria. Diawali dengan membuat daftar kebutuhan fungsional P/L, lengkap dengan ID dan penjelasan jika perlu.>

User Story : <Menggunakan format user stories untuk mendefinisikan kebutuhan tetapi memastikan setiap user story memiliki deskripsi yang menerangkan fungsi dan konteks fungsionality.

Contoh: "Sebagai [pengguna], saya ingin [fitur] sehingga [manfaat]."

Acceptance Criteria : <Kriteria penerimaan untuk setiap user story yang berfungsi sebagai batasan yang jelas tentang kapan kebutuhan tersebut dianggap telah terpenuhi.>

KEBUTUHAN FUNGSIONAL : Menjelaskan apa yang harus dilakukan oleh sistem

Misalnya “ pengguna dapat melakukan login”

USER STORY : Memformulasikannya dari perspektif pengguna, contohnya :

Sebagai pengguna,

Saya ingin dapat login ke aplikasi,

Sehingga sata bisa mengakses data pribadi saya.

ACCEPTANCE CRITERIA : Menjabarkan syarat/ketentuan agar user story dianggap selesai

1. Pengguna dapat memasukkan username dan password
2. Sistem memverifikasi krefensial
3. Jika benar, pengguna masuk ke dashboard
4. Jika salah, muncul pesan kesalahan

User Story + Acceptance Criteria = Bentuk ringkas, komunikatif, dan teruji untuk mendefinisikan kebutuhan fungsional

3.2 Kebutuhan Non-Fungsional

<Standar berkaitan dengan kinerja sistem, keamanan, kemudahan penggunaan, dan kondisi lain yang harus dipenuhi, dianalisis dan dijelaskan sesuai dengan kebutuhan.>

ID	Parameter	Kebutuhan
	Availability	
	Reliability	
	Ergonomy	
	Portability	
	Memory	
	Response time	
	Safety	
	Security	
	Others 1: Bahasa komunikasi	Misalnya : semua tanya jawab harus dalam bahasa Indonesia
		Setiap layar harus mengandung logo PT Pos Indonesia

- *Catatan :*
- Availability : ketersediaan aplikasi, misalnya harus terus menerus beroperasi 7 hari perminggu, 24 jam per hari tanpa gagal
- Reliability : keandalan, misalnya tidak pernah boleh gagal(atau kegagalan yang ditolerir adalah …%) sehingga harus dipikirkan fault tolerant architecture. Biasanya hanya perlu untuk Critical Application yang jika gagal akan berakibat fatal.
- Ergonomy : kenyamanan pakai bagi pengguna
- Portability : kemudahan untuk dibawa dan dioperasikan ke mesin/sistem operasi/platform yang lain
- Memory : jika perhitungan kapasitas memori internal kritis (misalnya untuk SW yang harus dijadikan CHIPS dan ukurannya harus kecil
- Response time : Batasan waktu yang harus dipenuhi. Sangat penting untuk aplikasi Real Time. Contoh: "Aplikasi harus mampu menampilkan hasil dalam 4 detik", atau "ATM harus menarik kembali kartu yang tidak diambil dalam waktu 3 menit"
- Safety: yang menyangkut keselamatan manusia, misalnya untuk SW yang dipakai pada sistem kontrol di pabrik
- Security : aspek keamanan yang harus dipenuhi